

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Ingeniería de Software (Rediseño) · Aplicaciones Web · 5to Nivel

ENTREGA 1A — PLANIFICACIÓN Y DISEÑO

Período Académico	2025-2026 SPA
Asignatura	Aplicaciones Web
Tipo de Entrega	Grupal — Proyecto Fin de Curso
Semana de Entrega	Semana 5 — Diciembre 2025
Docente	Dr. Gleiston Guerrero
Repositorio GitHub	SGED_APPWEB

INTEGRANTES DEL EQUIPO

Nombre Completo	Cédula
ARCALLE GREFA DARWIN ORLANDO	1501234973
PALLO PINTO DANIEL ALEJANDRO	1250064290
VELEZ LOPEZ RICARDO ELIAS	1316400942

1. Descripción del Sistema Web Propuesto

Las instituciones deportivas enfrentan actualmente diversas dificultades relacionadas con la gestión y administración de la información deportiva de los estudiantes. En muchos casos, los procesos del registro de estudiantes, el control de asistencia, la administración de membresías, la gestión de equipamiento deportivo, el seguimiento del rendimiento de los jugadores durante entrenamientos y partidos y generación de reportes se realizan mediante herramientas dispersas, documentos físicos o sistemas poco integrados. Esta situación provoca retrasos en la actualización de información, duplicidad de datos, errores humanos y dificultades para acceder a información confiable y actualizada.

Con el propósito de mejorar estos procesos, se propone el desarrollo de un Sistema de Gestión para Escuela Deportiva (SGED) basado en una aplicación web que permita centralizar la información deportiva y administrativa en una única plataforma accesible desde cualquier dispositivo con conexión a internet. El sistema estará orientado a optimizar la gestión mediante la reducción de errores en el manejo de información y la disponibilidad inmediata de datos para la toma de decisiones.

1.1 ¿Qué problema resuelve?

Las escuelas de fútbol formativas en Ecuador —particularmente las de nivel intermedio y pequeño— operan con procesos completamente manuales: registros en hojas de cálculo, control de asistencia en papel, gestión de pagos en cuadernos físicos y comunicación informal con padres de familia. Según datos del Ministerio del Deporte Ecuador (2023), existen más de 1.200 escuelas de fútbol registradas en el país, de las cuales el 78% no utiliza ningún software especializado. Esto genera pérdida de información, dificultad para hacer seguimiento del rendimiento de los jugadores, retrasos en cobros y falta de visibilidad para entrenadores y administradores. ProFútbol resuelve esta problemática digitalizando completamente la operación de la escuela.

1.2 ¿Quiénes son los usuarios?

Rol	Responsabilidades
Administrador	Supervisa el sistema completo, gestiona usuarios, permisos y genera reportes globales.
Entrenador	Evalúa jugadores, registra estadísticas, genera plantillas de partido y planifica entrenamientos.
Recepcionista	Registra estudiantes, gestiona pagos, membresías y préstamos de equipamiento.
Estudiante	Jugador de la escuela; tiene perfil deportivo, asiste a entrenamientos y recibe evaluaciones.

1.3 ¿Por qué una aplicación web?

La solución web es la más adecuada para este contexto por tres razones fundamentales: (1) **Acceso multiplataforma**: entrenadores pueden registrar evaluaciones desde el campo con su celular, recepcionistas desde computadoras de escritorio y administradores desde cualquier lugar; (2) **Sin instalación**: no requiere comprar licencias ni instalar software en equipos de la escuela; (3) **Centralización de datos**: toda la información de estudiantes, pagos y estadísticas reside en un servidor centralizado, eliminando inconsistencias entre versiones de archivos locales.

1.4 Alcance de esta entrega

El sistema ProFútbol fue concebido en el semestre 2025 SPA como un sistema integral. Para este semestre se implementarán los siguientes módulos núcleo que cubren el 70% de los requisitos funcionales prioritarios:

- ✓ Módulo de Autenticación y Gestión de Roles (login, registro, permisos por rol)
- ✓ Módulo de Gestión de Estudiantes (CRUD completo de perfiles deportivos)
- ✓ Módulo de Asistencia (registro por sesión de entrenamiento)
- ✓ Módulo de Evaluaciones (evaluación diaria por criterios con detalle por jugador)
- ✓ Módulo de Membresías y Pagos (control financiero básico)
- ✓ Módulo de Inventario de Equipamiento (gestión de préstamos)
- ✗ Módulo de Estadísticas de Partido con IA (fuera del alcance, se planifica para versión futura)

✗ Módulo de Reportes Avanzados (exportación PDF/Excel — fuera del alcance por restricción de tiempo)

■ El módulo de generación de plantillas con IA fue identificado como requisito de alto valor en el semestre anterior. Se excluye de esta entrega por su complejidad técnica, pero el modelo de base de datos lo contempla para desarrollo futuro sin cambios estructurales.

2. Revisión y Actualización de Requisitos del Semestre Anterior

2.1 Resumen del trabajo previo (semestre 2025 SPA)

En el semestre 2025 SPA el equipo desarrolló el proyecto en la asignatura de Ingeniería de Requisitos. Se utilizaron técnicas de elicitación de entrevistas con entrenadores y administradores de escuelas de fútbol locales, así como observación directa del proceso operativo. Se documentaron **18 requisitos funcionales** y **6 requisitos no funcionales** en un documento SRS (Software Requirements Specification) elaborado en Google Docs, con casos de uso modelados en draw.io. El sistema fue concebido originalmente como una aplicación de escritorio local.

2.2 Tabla de adaptación de requisitos

La siguiente tabla documenta el estado de cada requisito del semestre anterior frente al contexto web:

Cód.	Requisito original (2025 SPA)	Estado	Adaptación para la aplicación web
RF01	Registro de nuevos estudiantes con datos personales y deportivos	Mantiene	Formulario Angular multi-paso con validación en el componente TypeScript y consumo de API REST en Spring Boot
RF02	Autenticación de usuarios con usuario y contraseña	Adapta	Login Angular con JWT (JSON Web Tokens), hashing Argon2id en Spring Security, protección CSRF via token en headers
RF03	Gestión de roles de acceso diferenciado	Adapta	Guards de Angular + interceptor HTTP para token JWT + Spring Security Authorization en cada endpoint REST
RF04	Registro de asistencia diaria de estudiantes	Adapta	API REST para registro por sesión; soporte RFID mediante endpoint HTTP
RF05	Evaluación de rendimiento por criterios personalizados	Mantiene	Formulario web con calificación 1-10 por criterio y observaciones
RF06	Generación de plantillas de partido (alineación)	Adapta	Interfaz drag-and-drop web para asignar posiciones a jugadores
RF07	Registro de estadísticas por partido	Mantiene	Formulario web post-partido por jugador
RF08	Gestión de membresías con fechas y estados	Mantiene	CRUD web con cálculo automático de estado (activa/vencida)
RF09	Registro de pagos con saldo pendiente	Mantiene	Formulario de pago con actualización de saldo en tiempo real
RF10	Inventario de equipamiento deportivo	Mantiene	CRUD web con control de disponibilidad por unidad
RF11	Gestión de préstamos de equipamiento	Mantiene	Registro web con estados: pendiente/aprobado/devuelto
RF12	Gestión de horarios de entrenamiento recurrentes	Adapta	Formulario web con días de semana y generación automática de sesiones
RF13	Registro de lesiones de estudiantes	Mantiene	Formulario web con historial de lesiones por estudiante

RF14	Notificaciones a usuarios	Adapta	Notificaciones en-app (base de datos); email via SMTP para membresías
RF15	Generación de reportes de asistencia	Adapta	Vista web filtrable con exportación a PDF mediante librería del backend
RF16	Reporte de pagos y morosidad	Adapta	Dashboard web con indicadores de saldo pendiente por estudiante
RF17	Instalación local en computadora de la escuela	Fuera alcance	Reemplazado por despliegue web accesible por navegador
RF18	Generación de plantillas con IA	Fuera alcance	Alta complejidad; BD contempla la entidad; versión futura

2.3 Nuevos requisitos específicos del contexto web

ID	Nombre	Descripción
RNF-WEB-01	Compatibilidad de navegadores	La aplicación debe ser accesible desde Chrome 120+, Firefox 120+ y Edge 120+
RNF-WEB-02	Tiempo de respuesta	Cualquier página debe cargar en menos de 3 segundos en conexión de 10 Mbps
RNF-WEB-03	Diseño responsivo	La interfaz debe ser usable en pantallas de 320px a 1440px de ancho
RNF-WEB-04	Comunicación segura	Todas las comunicaciones deben realizarse sobre HTTPS en producción
RF-WEB-01	Autenticación web	El sistema debe implementar login web con gestión de sesiones y timeout automático
RF-WEB-02	Control de acceso por rol	Cada ruta del sistema debe validar el rol del usuario antes de servir el contenido
RF-WEB-03	Protección de comunicación	Todas las peticiones al backend deben incluir token JWT válido en el header Authorization

3. Lista Consolidada de Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-01	Registro de estudiantes	Alta	El recepcionista puede crear un perfil de estudiante con nombre, cédula, fecha de nacimiento, posición y datos de contacto. El perfil aparece en el listado de estudiantes activos.
RF-02	Autenticación de usuarios	Alta	Un usuario con credenciales válidas accede al sistema y es redirigido al dashboard correspondiente a su rol. Credenciales inválidas muestran mensaje de error sin revelar cuál campo es incorrecto.
RF-03	Gestión de roles y permisos	Alta	Cada ruta del sistema devuelve HTTP 403 si el usuario autenticado no tiene el rol requerido. El menú de navegación muestra solo las opciones del rol del usuario.
RF-04	Registro de asistencia	Alta	El entrenador puede registrar asistencia para cada estudiante en una sesión (Presente / Tardanza / Ausente). El sistema calcula y muestra el porcentaje de asistencia del estudiante.
RF-05	Evaluaciones diarias	Alta	El entrenador puede crear una evaluación para una sesión con criterios (Rendimiento, Estado Físico, Actitud) y calificar a cada estudiante de 1 a 10. La evaluación queda registrada con fecha y hora.
RF-06	Gestión de membresías	Alta	El recepcionista puede crear/renovar membresías con fecha inicio, fecha fin y monto. El sistema calcula automáticamente el estado (Activa/Vencida/Suspendida) y lo actualiza diariamente.

RF-07	Registro de pagos	Alta	El recepcionista puede registrar pagos indicando monto, método (Efectivo/Transferencia) y fecha. El sistema actualiza el saldo pendiente de la membresía automáticamente.
RF-08	Inventario de equipamiento	Media	El recepcionista puede ver el inventario de equipamiento con cantidad total y disponible. Al registrar un préstamo, la cantidad disponible disminuye; al devolver, aumenta.
RF-09	Préstamos de equipamiento	Media	El recepcionista puede registrar préstamos y devoluciones. El sistema registra fecha de préstamo y devolución y cambia el estado del préstamo (Pendiente/Devuelto).
RF-10	Gestión de horarios	Media	El entrenador puede crear horarios recurrentes (días de semana, hora, campo). El sistema genera automáticamente sesiones de entrenamiento para las próximas 4 semanas al crear el horario.
RF-11	Registro de lesiones	Media	El entrenador puede registrar lesiones de un estudiante con tipo, gravedad y fecha. El sistema muestra el historial de lesiones ordenado cronológicamente.
RF-12	Notificaciones en-app	Baja	El sistema genera notificaciones automáticas cuando una membresía está próxima a vencer (7 días antes) o cuando hay saldo pendiente superior a \$0. Las notificaciones aparecen en el panel de cada usuario.

4. Lista Consolidada de Requisitos No Funcionales

ID	Categoría	Descripción	Métrica de Verificación
RNF-01	Rendimiento	El tiempo de carga de cualquier página no debe superar 3 segundos.	Medido con Lighthouse en Chrome (simulación 4G); P95 ≤ 3s. Umbral: < 2s en conexión local.
RNF-02	Seguridad — Contraseñas	Las contraseñas deben almacenarse usando el algoritmo Argon2id con sal aleatoria.	Verificable inspeccionando la columna password_hash en la BD: debe iniciar con \$argon2id\$.
RNF-03	Usabilidad — Responsividad	La interfaz debe ser usable en pantallas de 320px a 1440px de ancho sin pérdida de funcionalidad.	Verificado con DevTools de Chrome en resoluciones 375px (móvil), 768px (tablet) y 1280px (desktop).
RNF-04	Disponibilidad	El sistema debe estar disponible y funcional durante las demostraciones del semestre.	Verificado por el docente durante las semanas 16 y 17. Tiempo de respuesta < 5s en demo.
RNF-05	Seguridad — Autenticación	Las sesiones deben tener tiempo de expiración de 120 minutos de inactividad.	Un usuario inactivo 120 min es redirigido al login. El token de sesión es invalidado en servidor.
RNF-06	Seguridad - Comunicación	Todas las peticiones al backend deben incluir token JWT válido en el header Authorization.	Solicitudes sin token o con token expirado reciben HTTP 401. Solicitudes con token válido pero sin permisos reciben HTTP 403.
RNF-07	Mantenibilidad	El código debe seguir el patrón MVC con separación clara de responsabilidades.	Revisión de código: no debe existir lógica SQL directa en vistas ni HTML en controladores.
RNF-08	Compatibilidad	La aplicación debe funcionar correctamente en Chrome 120+, Firefox 120+ y Edge 120+.	Pruebas manuales de funcionalidades core en los tres navegadores antes de la defensa.

5. Arquitectura de la Aplicación Web

5.1 Diagrama C4 Nivel 1 — Contexto del Sistema

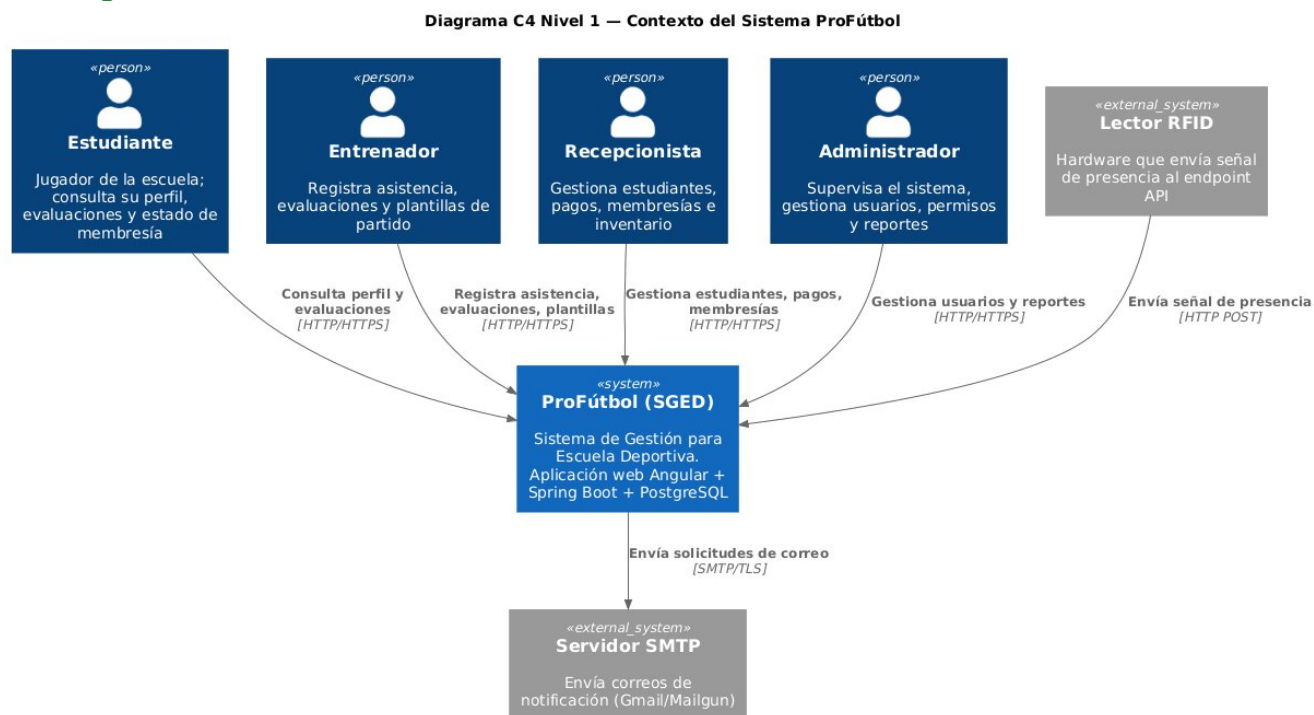


Figura 1 Contexto del sistema ProFútbol

5.2 Diagrama C4 Nivel 2 — Contenedores

Diagrama C4 Nivel 2 — Contenedores del Sistema ProFútbol

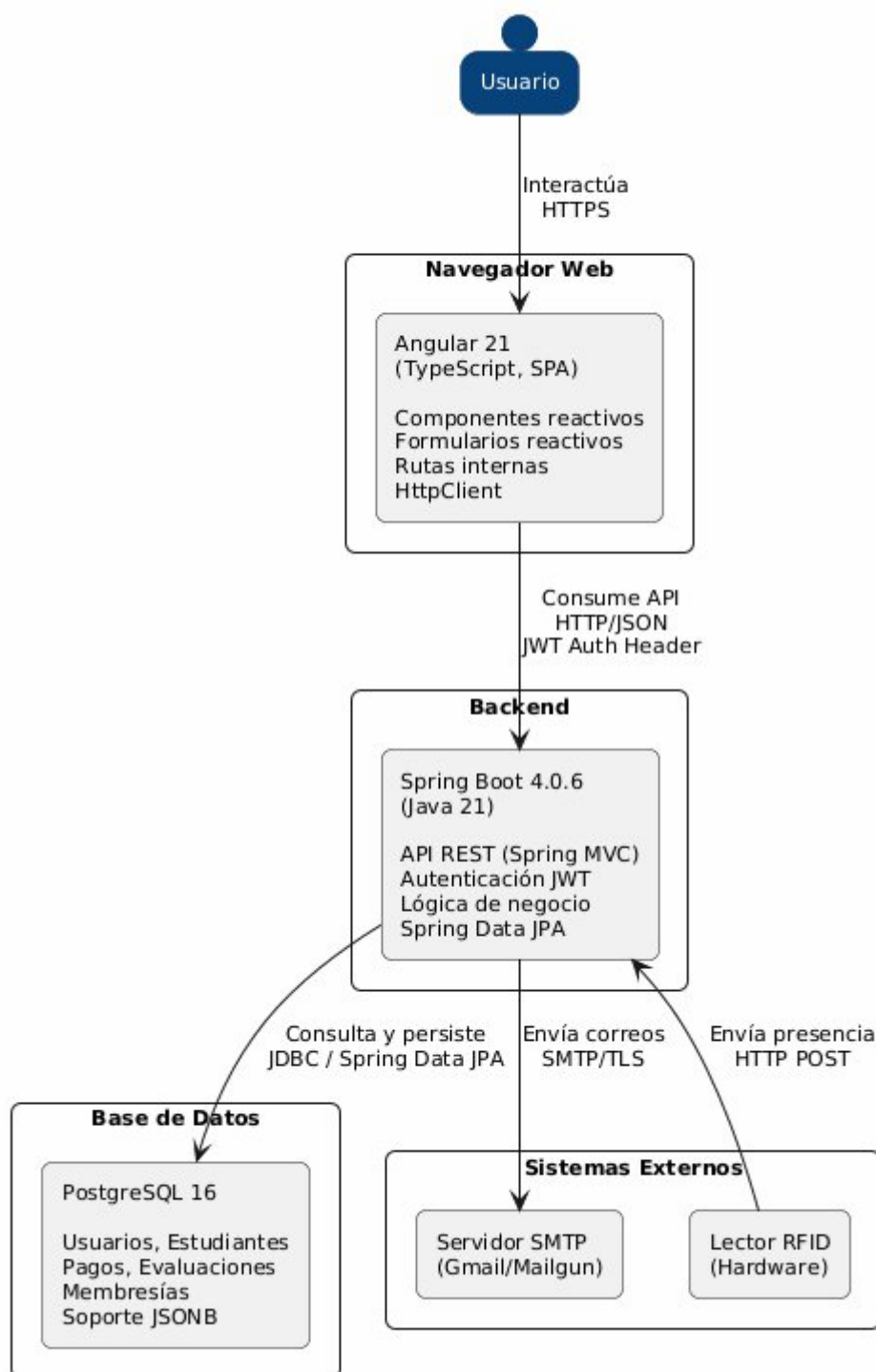


Figura 2 Diagrama de contenedores del sistema ProFútbol

5.3 Descripción de Capas y Tecnologías

Capa	Tecnología	Justificación
Frontend	Angular 21 (TypeScript)	Framework SPA robusto con tipado estático, inyección de dependencias, formularios reactivos y HttpClient nativo para consumo de APIs REST. Comunicación directa con Spring Boot vía JSON.
Framework CSS	Bootstrap 5.3 (o Angular Material)	Componentes responsivos probados, grilla flexible y documentación extensa. El equipo tiene experiencia previa, reduciendo la curva de aprendizaje.
Backend	Java 21 con Spring Boot 4.0.6	Ver ADR-001 para justificación completa. Spring Boot proporciona JPA (ORM), Spring Security (autenticación/autorización), Spring MVC (API REST) y servidor Tomcat embebido.
Base de datos	PostgreSQL 16	Motor relacional avanzado con soporte de JSONB, arrays nativos, extensiones, transacciones ACID. Sin costos de licencia. Rendimiento superior en consultas con múltiples JOINS.
Servidor web	Apache Tomcat (embebido en Spring Boot)	No requiere servidor externo. En producción se puede usar Nginx como reverse proxy frente a Tomcat.
Control de versiones	Git + GitHub	Repositorio compartido del equipo con política de ramas: main (producción), develop (integración), feature/xxx (desarrollo).
Despliegue	IntelliJ IDEA / VS Code (desarrollo) / Docker (producción)	Desarrollo local con IDE y JDK 21. Producción en Docker o servicios cloud como Heroku/Railway/Render.

5.4 ADR-001 — Decisión de Tecnología Principal

Título	ADR-001: Selección del stack tecnológico del servidor
Estado	Aceptado
Contexto	El equipo necesita elegir el stack tecnológico del lado del servidor para implementar el sistema ProFútbol como aplicación web. Restricciones: (1) el tiempo disponible es de 10 semanas de desarrollo efectivo; (2) el sistema requiere una API REST que Angular consuma vía HTTP/JSON; (3) se necesita autenticación segura por roles; (4) el motor de base de datos es PostgreSQL; (5) el docente indicó que PHP 8.2, ASP.NET Core 8 y Java/JSP son las opciones válidas para el backend.
Opciones consideradas	Opción A: PHP 8.2 con Laravel 11 — framework full-stack, hosting económico. Opción B: ASP.NET Core 8 — robusto, tipado estático, requiere hosting dedicado. Opción C: Java 21 con Spring Boot 4.0.6 — ecosistema enterprise, JPA para ORM, Angular se comunica nativamente con backend Java por HTTP/JSON.
Decisión	Se elige la Opción C: Java 21 con Spring Boot 4.0.6. Razones: (1) Spring Boot simplifica la creación de APIs REST con Spring MVC y accede a PostgreSQL via Spring Data JPA; (2) Angular (frontend) se comunica nativamente con el backend Java a través de peticiones HTTP/JSON, sin capas intermedias; (3) Spring Security proporciona autenticación JWT robusta con soporte para roles; (4) IntelliJ IDEA Community Edition es gratuito; (5) PostgreSQL es el motor elegido (sin costos de licencia, soporte JSONB, rendimiento superior en consultas complejas).

Consecuencias positivas	• Ecosistema enterprise con documentación extensa. • Tipado estático (Java) reduce errores en runtime. • Angular + Spring Boot forman un stack coherente (TypeScript + Java, ambos tipados). • Spring Security maneja JWT, roles y protección de endpoints de forma integrada. • JPA/Hibernate mapea entidades Java a tablas PostgreSQL sin SQL manual.
Consecuencias negativas	• • Curva de aprendizaje de Spring Boot (~2 semanas). Mitigación: Spring Initializr genera la estructura base, documentación oficial de Spring. • Requiere JDK 21 instalado. Mitigación: SDKMAN gestiona versiones de JDK fácilmente. • Hosting Java más costoso que hosting PHP compartido. Mitigación: Heroku/Railway/Render ofrecen tier gratuito para aplicaciones Java. • Configuración inicial más compleja que Laravel. Mitigación: Spring Boot auto-configura la mayoría de componentes.

6. Modelo de Base de Datos

6.1 Modelo Entidad-Relación

El sistema ProFútbol cuenta con 21 entidades principales. A continuación se presenta el resumen de relaciones entre las entidades más importantes. [El diagrama MER completo debe incluirse como PNG exportado de dbdiagram.io]

Entidad A	Entidad B	Cardinalidad	Descripción
Persona	Estudiante	1:1	Estudiante hereda/extiende de Persona
Persona	Entrenador	1:1	Entrenador hereda/extiende de Persona
Persona	Recepcionista	1:1	Recepcionista hereda/extiende de Persona
Entrenador	HorarioEntrenamiento	1:N	Un entrenador crea muchos horarios
HorarioEntrenamiento	SesionEntrenamiento	1:N	Un horario genera muchas sesiones
SesionEntrenamiento	Asistencia	1:N	Una sesión tiene asistencia de muchos estudiantes
SesionEntrenamiento	EvaluacionDiaria	1:1	Una sesión tiene una evaluación
EvaluacionDiaria	CriterioEvaluacion	1:N	Una evaluación tiene varios criterios
CriterioEvaluacion	DetalleEvaluacion	1:N	Un criterio tiene la nota de muchos estudiantes
Estudiante	Membresia	1:N	Un estudiante puede tener varias membresías (histórico)
Membresia	Pagos	1:N	Una membresía puede tener varios pagos
Estudiante	PrestamoEquipamiento	1:N	Un estudiante puede tener varios préstamos
Equipamiento	PrestamoEquipamiento	1:N	Un equipamiento puede prestarse múltiples veces
Partido	PlantillaPartido	1:1	Un partido tiene una plantilla
PlantillaPartido	JugadorPlantilla	1:N	Una plantilla tiene varios jugadores

Tabla de relaciones entre entidades del sistema ProFútbol.

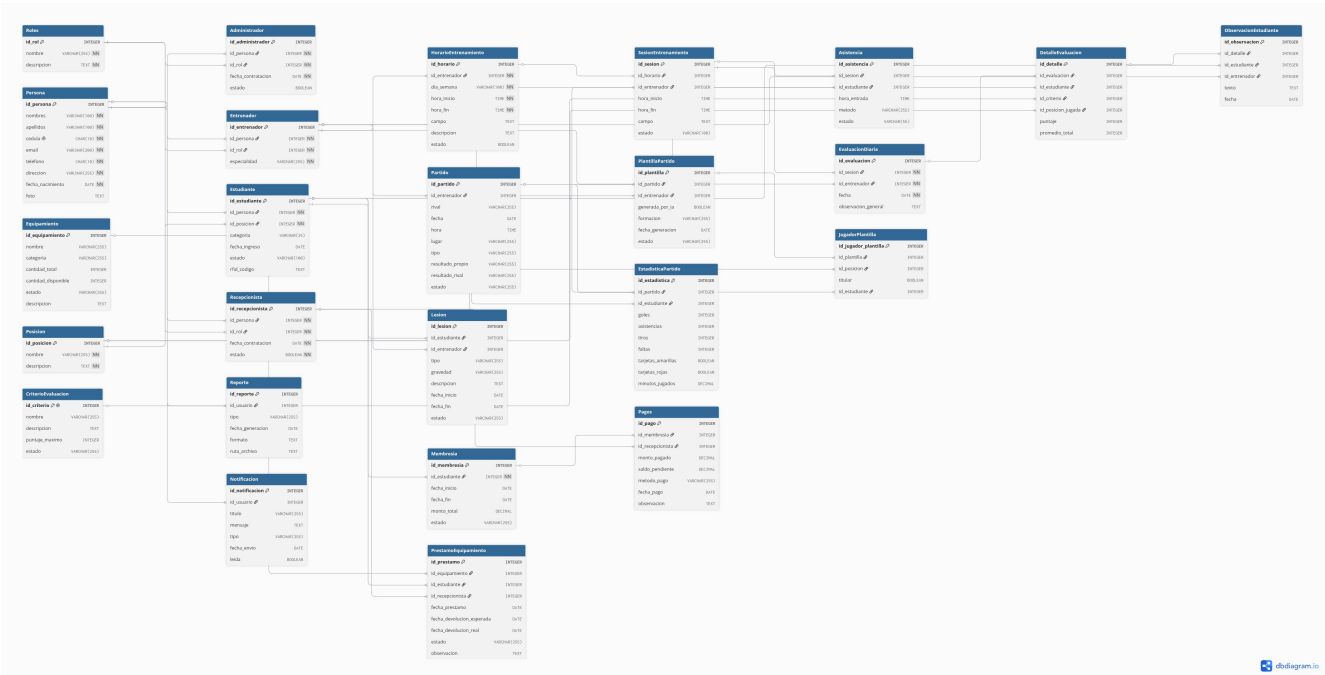


Figura 3 Diagrama Modelo Entidad-Relación

6.2 Diccionario de Datos (tablas principales)

Tabla: personas

Atributo	Tipo	Nulo	Restricción	Descripción
id	INT	No	PK, GENERATED BY DEFAULT AS IDENTITY	Identificador único
nombres	VARCHAR(100)	No	NOT NULL	Nombres completos
apellidos	VARCHAR(100)	No	NOT NULL	Apellidos completos
cedula	VARCHAR(13)	No	UNIQUE	Cédula ecuatoriana
email	VARCHAR(255)	No	UNIQUE	Correo electrónico para login
password_hash	VARCHAR(255)	No	NOT NULL	Hash Argon2id de la contraseña
telefono	VARCHAR(15)	Sí	-	Teléfono de contacto
rol	VARCHAR(20) CHECK (rol IN ('admin','entrenador','recepcionista','estudiante'))	No	DEFAULT 'estudiante'	Rol del usuario
creado_en	TIMESTAMP	No	DEFAULT CURRENT_TIMESTAMP	Fecha de registro

Tabla: estudiantes

Atributo	Tipo	Nulo	Restricción	Descripción
id	INT	No	PK, GENERATED BY DEFAULT AS IDENTITY	Identificador único
persona_id	INT	No	FK → personas.id	Referencia a datos personales
posicion_id	INT	Sí	FK → posiciones.id	Posición preferida en el campo
rfid_codigo	VARCHAR(50)	Sí	UNIQUE	Código del chip RFID para asistencia

fecha_nacimiento	DATE	No	NOT NULL	Fecha de nacimiento del jugador
numero_camiseta	SMALLINT	Sí	-	Número asignado de camiseta
activo	BOOLEAN	No	DEFAULT 1	1=activo, 0=inactivo/retirado

Tabla: sesiones_entrenamiento

Atributo	Tipo	Nulo	Restricción	Descripción
id	INT	No	PK, GENERATED BY DEFAULT AS IDENTITY	Identificador único de la sesión
horario_id	INT	No	FK → horarios_entrenamiento.id	Horario del que se generó
fecha	DATE	No	NOT NULL	Fecha del entrenamiento
hora_inicio	TIME	No	NOT NULL	Hora de inicio de la sesión
campo	VARCHAR(50)	Sí	-	Nombre del campo/cancha
estado	VARCHAR(20) CHECK (estado IN ('programada','en curso','finalizada','cancelada'))	No	DEFAULT 'programada'	Estado de la sesión

Tabla: membresias

Atributo	Tipo	Nulo	Restricción	Descripción
id	INT	No	PK, GENERATED BY DEFAULT AS IDENTITY	Identificador único
estudiante_id	INT	No	FK → estudiantes.id	Estudiante al que pertenece
fecha_inicio	DATE	No	NOT NULL	Inicio de la membresía
fecha_fin	DATE	No	NOT NULL	Fecha de vencimiento
monto_total	DECIMAL(8,2)	No	NOT NULL	Valor total de la membresía
saldo_pendiente	DECIMAL(8,2)	No	DEFAULT 0	Saldo por cobrar
estado	VARCHAR(20) CHECK (estado IN ('activa','vencida','suspendida'))	No	DEFAULT 'activa'	Estado calculado automáticamente

6.3 Script DDL de Creación

El script SQL completo está en el repositorio en **database/schema.sql**. Fragmento de las tablas principales:

```
-- ProFútbol Database Schema v1.0
-- Motor: PostgreSQL 16
CREATE DATABASE IF NOT EXISTS profutbol CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE profutbol;

CREATE TABLE personas (
  id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  nombres VARCHAR(100) NOT NULL,
  apellidos VARCHAR(100) NOT NULL,
  cedula VARCHAR(13) NOT NULL UNIQUE,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  telefono VARCHAR(15) DEFAULT NULL,
  rol VARCHAR(20) CHECK (rol IN ('admin','entrenador','repcionista','estudiante')) DEFAULT
'estudiante',
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  actualizado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX idx_email ON personas(email);
CREATE INDEX idx_rol ON personas(rol);

CREATE TABLE posiciones (
  id SMALLINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE estudiantes (
  id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  persona_id INTEGER NOT NULL,
  posicion_id SMALLINT DEFAULT NULL,
  rfid_codigo VARCHAR(50) DEFAULT NULL UNIQUE,
  fecha_nacimiento DATE NOT NULL,
  numero_camiseta SMALLINT DEFAULT NULL,
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  FOREIGN KEY (persona_id) REFERENCES personas(id) ON DELETE CASCADE,
  FOREIGN KEY (posicion_id) REFERENCES posiciones(id) ON DELETE SET NULL
);
CREATE INDEX idx_rfid ON estudiantes(rfid_codigo);

CREATE TABLE membresias (
  id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  estudiante_id INTEGER NOT NULL,
  fecha_inicio DATE NOT NULL,
  fecha_fin DATE NOT NULL,
  monto_total DECIMAL(8,2) NOT NULL,
  saldo_pendiente DECIMAL(8,2) NOT NULL DEFAULT 0.00,
  estado VARCHAR(20) CHECK (estado IN ('activa','vencida','suspendida')) NOT NULL DEFAULT
'activa',
  creado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (estudiante_id) REFERENCES estudiantes(id) ON DELETE CASCADE
);
CREATE INDEX idx_estado ON membresias(estado);
CREATE INDEX idx_fecha_fin ON membresias(fecha_fin);
-- (Ver database/schema.sql en repositorio para script completo)
```

7. Diseño de Interfaz: Wireframes

Los wireframes fueron elaborados en Figma y se presentan a continuación. [Los wireframes reales en PNG exportados de Figma deben reemplazar las descripciones de cada pantalla].

WF-01 — Página de Login

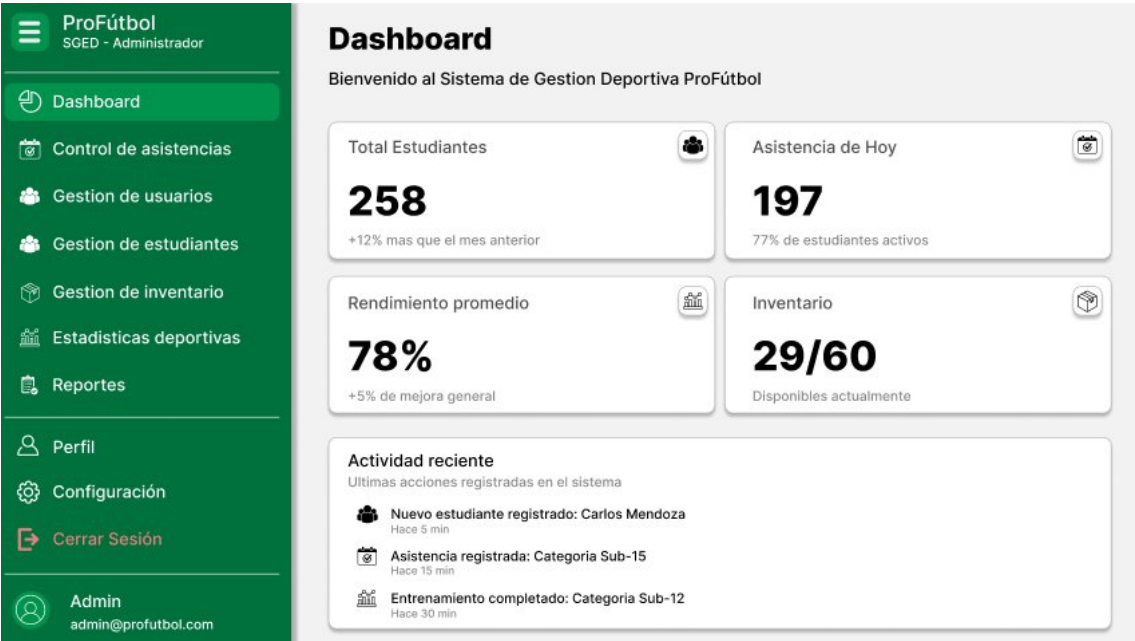
Rol que la ve	Todos los usuarios (público)
Elementos y función	Formulario centrado con logo ProFútbol, campo Email, campo Contraseña, botón "Ingresar" (verde), enlace "¿Olvidé mi contraseña?". Mensajes de error en rojo bajo el campo correspondiente. Fondo con imagen de cancha de fútbol en baja opacidad.
Flujo de navegación	Entrada: acceso directo desde URL raíz. Salida: redirige al dashboard del rol del usuario autenticado.



Pantalla WF-01 — Página de Login

WF-02 — Dashboard del Entrenador

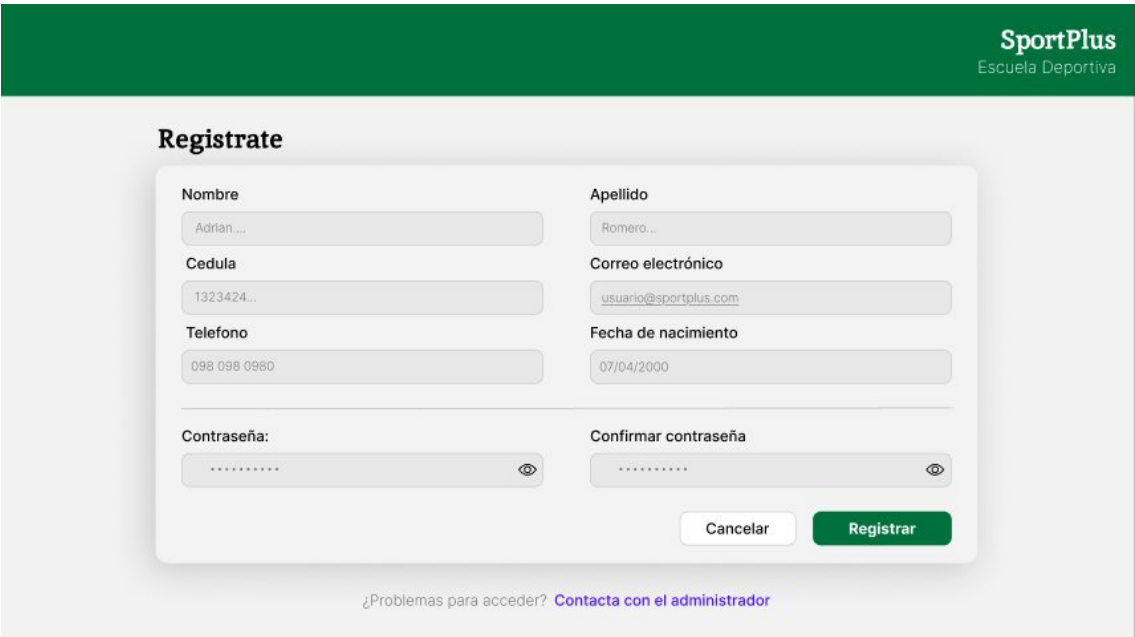
Rol que la ve	Entrenador autenticado
Elementos y función	Barra lateral izquierda con navegación: Inicio, Estudiantes, Horarios, Sesiones, Evaluaciones, Plantillas, Estadísticas. Área principal: tarjetas de resumen (próxima sesión, pendientes de evaluar, alertas de lesiones). Lista de próximas 3 sesiones con botón "Registrar asistencia".
Flujo de navegación	Entrada: login exitoso con rol Entrenador. Salida: cualquier módulo de la barra lateral.



Pantalla WF-02 — Dashboard del Entrenador

WF-03 — Registro de Nuevo Usuario

Rol que la ve	Todos los usuarios (público)
Elementos y función	Formulario con campos: Nombre, Apellido, Cédula, Correo Electrónico, Teléfono, Fecha de nacimiento, Contraseña, Confirmar contraseña. Acciones (Cancelar/Guardar).
Flujo de navegación	Entrada: Pagina Login → Regístrate. Salida: formulario con campos a rellenar



Pantalla WF-03 — Registro de Nuevo Usuario

WF-04 — Gestión de estudiantes

Rol que la ve	Entrenador autenticado
Elementos y función	barra de búsqueda, filtro y un botón de agregar nuevo estudiantes, muestra lista de estudiantes se pueden realizar 3 cosas: Ver perfil, descargar pdf y eliminar
Flujo de navegación	Entrada: acceso directo desde URL raíz. Salida: redirige al <u>dashboard</u> del rol del usuario autenticado.



Pantalla WF-04 — Gestor de estudiantes

8. Plan de Trabajo: Cronograma e Hitos

8.1 Cronograma semanal (Semanas 5–17)

Semana	Tarea Principal	Responsable	Entregable Asociado
5	Entrega 1A: planificación y diseño	Todo el equipo	PDF Entrega 1A en LMS (Sumativa #5)
6	Configuración entorno + estructura Spring Boot + rutas REST + autenticación JWT	Integrante 1 (Líder)	Repositorio con estructura inicial y login funcional
7	PE Unidad II: Backend Java/Spring Boot + CRUD Personas y Estudiantes	Todo el equipo	PE Unidad II (Sumativa #6)
8	Módulo de Horarios y Sesiones de Entrenamiento + Relaciones ORM	Integrante 2 (Backend)	Modelos y controladores de horarios listos
9	Evaluación Parcial 1 Individual + Correcciones de retroalimentación	Individual	EP Primer Corte (Sumativa #7)
10	Módulo de Asistencia (registro manual + endpoint RFID) + Frontend responsivo	Integrantes 1 y 3	Avance PFC Módulo datos (Sumativa #8)
11	Módulo de Evaluaciones Diarias (criterios + detalles) + PE Unidad III	Todo el equipo	PE Unidad III (Sumativa #9)
12	Módulo de Membresías y Pagos + integración de capas	Integrante 2 + 3	CRUD de membresías y pagos funcional
13	Módulo de Inventario y Préstamos + Exposición Backend	Integrante 2	Exposición Back-End (Sumativa #12)
14	Notificaciones en-app + Panel de reportes básico + Pruebas integración	Todo el equipo	Sistema funcional end-to-end
15	Correcciones finales + Documentación + Entrega 1B	Todo el equipo	Entrega 1B en LMS (Sumativa #15)
16	PE Unidad IV + Pruebas de aceptación + Despliegue final	Todo el equipo	PE Unidad IV (Sumativa #16)

17	Defensa oral del PFC + Evaluación Final	Todo el equipo	EP Segundo Corte + Defensa (Sumativa #17)
----	---	----------------	---

Las semanas marcadas en dorado corresponden a hitos sumativas.

8.2 Asignación de roles del equipo

Integrante	Rol Principal	Responsabilidades
Pallo	Líder Técnico / Full Stack	Coordinación del equipo, revisiones de código, decisiones de arquitectura, configuración del entorno, módulo de autenticación, integración de capas.
Velez	Desarrollador Backend	Modelos JPA (entidades Java), controladores REST, migraciones SQL, relaciones ORM via Hibernate
Arcalle	Desarrollador Frontend	Componentes Angular, Bootstrap 5, formularios reactivos, responsividad, servicios HTTP

■ Los roles no son exclusivos: cada integrante debe tener commits en todos los módulos. El historial de GitHub es evidencia de participación grupal.

9. Estructura del Repositorio Git

El repositorio del proyecto está disponible en: https://github.com/DarwinSM21/SGED_APPWEB.git

Ruta / Archivo	Contenido
README.md	Descripción del proyecto, instrucciones de instalación, integrantes, URL del sistema desplegado
docs/entrega-1a.pdf	Este informe en PDF
docs/diagramas/	Diagramas C4 Nivel 1 y 2 (PNG), MER (PNG) y wireframes de Figma
docs/adr/ADR-001-tecnologia.md	Architecture Decision Record de la selección tecnológica
database/schema.sql	Script DDL completo de creación de la base de datos PostgreSQL
.gitignore	Excluye: vendor/, .env, node_modules/, *.log, storage/*.log
.env.example	Variables de entorno con nombres pero sin valores reales
backend/src/	Entidades, Controladores, Servicios y Seguridad de Spring Boot
frontend/src/	Componentes Angular de la aplicación
public/	Assets públicos: CSS, JS, imágenes

■ Todos los integrantes del equipo deben tener al menos 1 commit en el repositorio antes de la fecha de entrega. El primer commit puede ser la creación de la estructura de carpetas y el README.

10. Referencias Bibliográficas

1. Sommerville, I. (2015). *Ingeniería del software* (10.a ed.). Pearson Education. ISBN: 978-0-13-394303-0
2. Spring Boot Documentation (2024). Spring Boot Reference Documentation. VMware.
<https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
3. Brown, S. (2023). *The C4 model for visualising software architecture*. Leanpub. Recuperado de <https://c4model.com/>
4. Washizaki, H. (Ed.). (2024). *SWEBOK Guide v4.0*. IEEE Computer Society. Recuperado de <https://www.computer.org/education/bodies-of-knowledge/software-engineering>
5. Walls, C. (2023). *Spring Boot in Action* (2.a ed.). Manning Publications. ISBN: 978-1-61729-494-7
6. Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.a ed.). Addison-Wesley. ISBN: 978-0-13-688080-1
7. Ministerio del Deporte Ecuador (2023). *Estadísticas de escuelas deportivas registradas 2023*. Gobierno del Ecuador. Recuperado de <https://www.deporte.gob.ec>
8. PostgreSQL Global Development Group (2024). PostgreSQL 16 Documentation.
<https://www.postgresql.org/docs/16/>
9. Prohaczewski, G. (2023). *Angular Up and Running: Learn to Build and Deploy Angular Applications*. O'Reilly Media. ISBN: 978-1-098-12499-4